《人工智能基础方法与应用》 课堂作业

Seoul Bike Sharing Demand · 线性回归建模与评估

基于线性回归的
共享单车租赁需求预测

第一部分：作业要求逐条交付

第二部分：超额研究（探索性分析 · 交叉验证 · 统计检验 · 改进实验）

姓 名： 孙承泽

学 号： 2253710052

提交日期： 2026 年 10 月 14 日

数据来源： UCI Seoul Bike Sharing Demand（作业提供子集：训练 100 条 / 测试 60 条）

交付形式： PDF 电子报告，另附可复现代码、图件与结果文件

摘要

我在 100 条首尔共享单车运营数据上训练了 9 特征线性回归模型 $y = b + \Sigma\, w_i x_i$：对类别特征做哑变量编码、对连续特征做标准化，按最小二乘正规方程求解，并用梯度下降独立复算；原始尺度参数 $b = 1055.483$ 与各 w_i 见正文。模型在训练集（$R^2 = 0.506$、RMSE = 471.0）与测试集（$R^2 = 0.345$、RMSE = 508.2）上均优于均值基线，误差降低 19.5%。此外还做了探索性数据分析、参数显著性检验与共线性诊断、残差诊断，以及四组改进实验：加入温度二次项与“温度×季节”交互后，测试集 R^2 提升至 0.461。

关键词：线性回归；最小二乘；正规方程；梯度下降；特征标准化；共享单车需求预测

目 录

任务完成情况自查表 3

第一部分 · 作业要求交付 4

要求一 · 建立线性回归模型 5

要求二 · 数据处理、求解过程与参数 b、w_i 6

2.1 数据处理 6

2.2 模型求解过程 6

2.3 训练结果：参数 b 与 w_i 7

要求三 · 训练集与测试集表现分析 9

第二部分 · 超额研究 11

§4 探索性数据分析（补充） 12

§5 三种求解方法交叉验证与数值校验 14

§6 参数显著性检验与共线性诊断（补充） 15

§7 残差诊断与预测结构分析（补充） 17

§8 改进探索实验（补充） 18

§9 结论与展望 19

附录 A 完整代码（可复现） 20

任务完成情况自查表

交卷前我把作业的五项要求逐条核对了一遍，结果如下表：要求一至三在第一部分逐条交付，第二部分是延伸研究；要求四、要求五属于提交事项，我在表中一并注明。

#	作业要求	对应交付内容	位置
1	建立线性回归模型 $y = b + \Sigma w_i x_i$	完成。要求一：给出模型形式（式 1）与 9 个特征的完整定义（表 1）	第一部分 · 要求一
2	用 100 条训练数据训练模型，说明数据处理与求解过程，给出参数 b 与各 w_i	完成。要求二：数据处理（2.1 节，表 2、表 3）；求解过程（2.2 节，式 3–6、图 1）；参数结果（2.3 节，表 4）	第一部分 · 要求二
3	分析模型在训练集与测试集上的表现	完成。要求三：评价指标（式 7、式 8）、结果（表 5、图 2）与四点分析（3.3 节）	第一部分 · 要求三
4	提交时限：两周之内	完成。报告于 2026 年 10 月 14 日按期提交	—
5	提交 PDF 文档，以“姓名-学号-共享单车回归作业”命名，发送至助教邮箱	完成。文件按“孙承泽-2253710052-共享单车回归作业.pdf”命名，发送至 liushuaiyu@stu.xjtu.edu.cn	—

第一部分 · 作业要求交付

逐条回答作业要求：模型 → 数据处理与求解 → 参数 b、w_i → 训练/测试表现。
交付位置与前页自查表一致。

要求一 · 建立线性回归模型

我以 100 条首尔共享单车运营数据为训练样本建立模型，目标是预测共享单车租赁数量。记租赁数量为 y，选取 9 个特征 x1–x9（气象 5 个 + 季节哑变量 3 个 + 节假日 1 个，定义见表 1），建立如下线性回归模型：

$$y = b + \sum_{i=1}^{9} w_i x_i = b + w_1 x_1 + w_2 x_2 + \cdots + w_9 x_9$$

式 (1) 线性回归模型（b 为截距，w_i 为回归系数）

模型共含 10 个待估参数 (b, w1, ⋯, w9)。其中连续特征直接取值；类别特征 “季节” 含 4 个取值，以 “冬季” 为基准设 3 个哑变量； “节假日” 为二值变量，设 1 个哑变量（1 = 节假日，0 = 非节假日）。各变量的含义与取值见表 1。

表 1 线性回归模型的特征与目标变量定义

变量	含义	类型	取值 / 说明
x1	温度 (°C)	连续	−15.3 ~ 32.4
x2	湿度 (%)	连续	15 ~ 97
x3	风速 (m/s)	连续	0 ~ 5.6
x4	降雨量 (mm)	连续	0 ~ 2.5
x5	降雪量 (cm)	连续	0 ~ 2.6
x6–x8	季节哑变量	0/1	春季/夏季/秋季 各 1 列，冬季为基准
x9	节假日	0/1	1 = 节假日，0 = 非节假日
y	租赁自行车数量（目标）	整数	0 ~ 2577

要求二 · 数据处理、求解过程与参数 b、w_i

下面分三小节：2.1 数据处理（哑变量编码与标准化）；2.2 求解过程（最小二乘 → 正规方程 → 实现）；2.3 训练结果（原始尺度参数 b 与全部 w_i）。训练只用作业提供的 100 条样本；测试集 60 条留到要求三的表现分析，全程不参与训练。

2.1 数据处理

- 步骤 1 · 数据检查：训练集与测试集均无缺失值、无重复样本，无需插补或去重；
- 步骤 2 · 哑变量编码：解决“虚拟变量陷阱”（见下），类别特征转为 0/1 列（表 2）；
- 步骤 3 · 连续特征标准化（式 2）：μ、σ 只由训练集估计，再原样套用到测试集（防止信息泄漏）；
- 步骤 4 · 拼接常数列（全为 1），得到 100×10 的设计矩阵 X。

关于哑变量编码：“季节”若有 4 个 0/1 指示变量，四列之和恒等于常数列 1，设计矩阵将秩亏、正规方程无唯一解。因此以“冬季”为基准类，只保留春、夏、秋 3 列，冬季的效应被吸收进截距 b；“节假日”同理只需 1 列。编码示例见表 2。

表 2 类别特征哑变量编码示例（前 3 条样本）

#	温度	湿度	风速	降雨	降雪	季节	节假日	春季	夏季	秋季	节假日
1	27.0	33	2.2	0	0	春季	非节假日	1	0	0	0
2	27.4	77	0.7	0	0	夏季	非节假日	0	1	0	0
3	9.7	56	1.3	0	0	秋季	非节假日	0	0	1	0

注：最后一列“节假日”为二元特征（1 = 节假日，0 = 非节假日）。编码后 100×9 的特征矩阵与常数列拼接，得到 100×10 的设计矩阵 X，其秩为 10（满列秩），保证最小二乘解唯一。

关于标准化：连续特征量纲差异悬殊（温度以十计、降雨以 0.1 计），直接求解会让设计矩阵条件数过大。对 5 个连续特征做标准化（式 2），使各列处于同一尺度，条件数由 497.4 降至 12.1，数值求解更稳定。

$$z_j = \frac{x_j - \mu_j}{\sigma_j}, \qquad \hat{y} = b^* + \sum_{j=1}^{9} w_j^* z_j$$

式 (2) 连续特征标准化（μ_j、σ_j 由训练集估计）

表 3 标准化所用统计量（仅由训练集 100 条数据计算）

连续特征	训练集均值 μ_j	训练集标准差 σ_j
温度 (°C)	11.078	12.368
湿度 (%)	56.890	20.546
风速 (m/s)	1.848	1.199
降雨量 (mm)	0.106	0.448
降雪量 (cm)	0.129	0.471

2.2 模型求解过程

求解即寻找使预测误差平方和最小的参数，等价于最小化目标函数（式 3）：

$$L(b,\mathbf{w})=\frac{1}{n}\sum_{i=1}^{n}(y_i-\hat{y}_i)^2,\qquad \hat{y}_i=b+\sum_{j=1}^{9}w_j x_{ij}$$

式 (3) 最小二乘目标函数（n = 100 为训练样本数）

对 b 与 w 求偏导并令其为零，得到正规方程（式 4），其解即最小二乘解；几何上它是目标向量 y 在设计矩阵列空间上的正交投影。

$$\frac{\partial L}{\partial b}=0,\ \frac{\partial L}{\partial \mathbf{w}}=0 \Longrightarrow \mathbf{X}^{\top}\mathbf{X}\mathbf{w}=\mathbf{X}^{\top}\mathbf{y} \Longrightarrow \mathbf{w}^{*}=(\mathbf{X}^{\top}\mathbf{X})^{-1}\mathbf{X}^{\top}\mathbf{y}$$

式 (4) 正规方程及最小二乘闭式解

数值实现上不直接求逆，而以 np.linalg.solve 解线性方程组 (X^T X) w = X^T y，避免求逆带来的额外误差；同时用 scikit-learn 的 LinearRegression 对照验证（两者差异见第二部分 §5）。为检查求解的收敛性，我又用批量梯度下降从零实现了同一问题的求解（式 5），学习率 η = 0.1、迭代 30000 次，损失曲线见图 1。

$$\mathbf{w}\leftarrow\mathbf{w}-\eta\nabla_{\mathbf{w}}L,\qquad \eta=0.1,\ \ 30000\ \text{iterations}$$

式 (5) 批量梯度下降参数更新规则（全量 100 样本每步计算梯度）

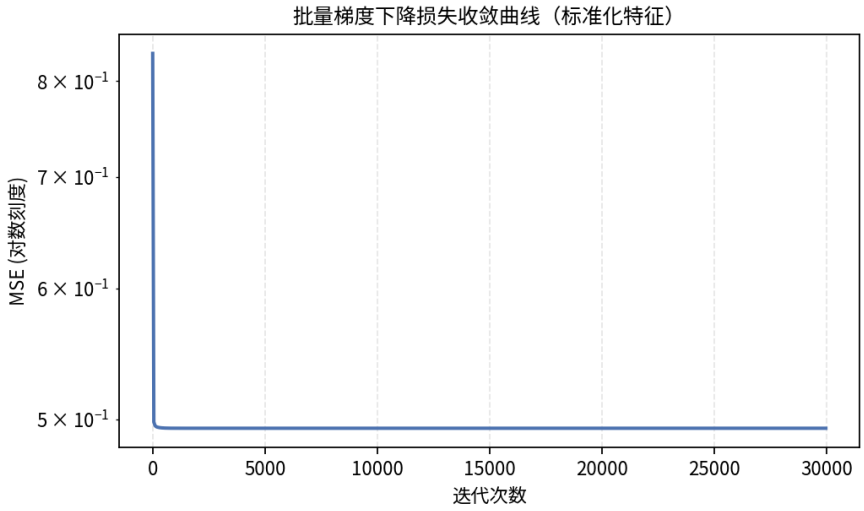


图 1 批量梯度下降损失收敛曲线（纵轴为对数刻度的 MSE，横轴为迭代次数）

此时所得参数处于标准化尺度；按式 2 的线性关系做精确反变换（式 6），即可得到原始物理单位下的 b 与 w_i，反变换的数值校验见第二部分 §5。

$$w_j=\frac{w_j^{*}}{\sigma_j},\qquad b=b^{*}-\sum_{j=1}^{5}\frac{w_j^{*}\,\mu_j}{\sigma_j}$$

式 (6) 参数由标准化尺度反变换回原始尺度

2.3 训练结果：参数 b 与 w_i

在 100 条训练数据上学得的原始尺度模型为（保留 3 位小数）：

y = 1055.483 + 39.199·x1(温度) − 11.039·x2(湿度) − 61.311·x3(风速) − 233.790·x4(降雨) − 39.490·x5(降雪) + 97.676·I(春季) − 220.914·I(夏季) − 33.487·I(秋季) − 269.995·I(节假日)

表 4 参数估计结果（原始尺度，单位：辆）

参数	含义	取值
b	截距（基准：冬季、非节假日、0 °C、无风无雨雪）	+1055.483
w1	温度 (°C)	+39.199
w2	湿度 (%)	-11.039
w3	风速 (m/s)	-61.311
w4	降雨量 (mm)	-233.790
w5	降雪量 (cm)	-39.490
w6	季节=春季	+97.676
w7	季节=夏季	-220.914
w8	季节=秋季	-33.487
w9	节假日	-269.995

注：全部 10 个参数的标准化尺度值、标准误与显著性检验见第二部分 §6（表 9）。

要求三 · 训练集与测试集表现分析

3.1 评价指标

采用 R^2（对需求方差的解释比例）、RMSE（与大误差相关、量纲为“辆”）、MAE（平均绝对误差）与 MAPE 四个指标；其中数据含大量近零需求样本，MAPE 仅作参考（原因见 3.3）。定义见式 7 与式 8。

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}, \qquad \mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

式 (7) R^2 与 RMSE 的定义

$$\mathrm{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|, \qquad \mathrm{MAPE} = \frac{100\%}{n}\sum_{i=1}^{n}\left|\frac{y_i - \hat{y}_i}{y_i}\right|$$

式 (8) MAE 与 MAPE 的定义

3.2 结果

表 5 模型在训练集、测试集与基线上的表现（R^2 越高越好，误差越低越好）

数据集 / 对照	R^2	调整 R^2	RMSE (辆)	MAE (辆)	MAPE
训练集 (n=100)	0.5061	0.4567	471.0	355.9	129.6%
测试集 (n=60)	0.3450	0.2271	508.2	387.9	122.0%
均值基线 (测试集)	-0.0105	—	631.2	500.6	212.1%
5 折交叉验证 (训练集)	0.3889 ± 0.1198	—	512.7	—	—

均值基线 = 始终预测训练集平均需求（670.9 辆）；5 折交叉验证在训练集内部进行，用于更稳健地估计泛化能力。

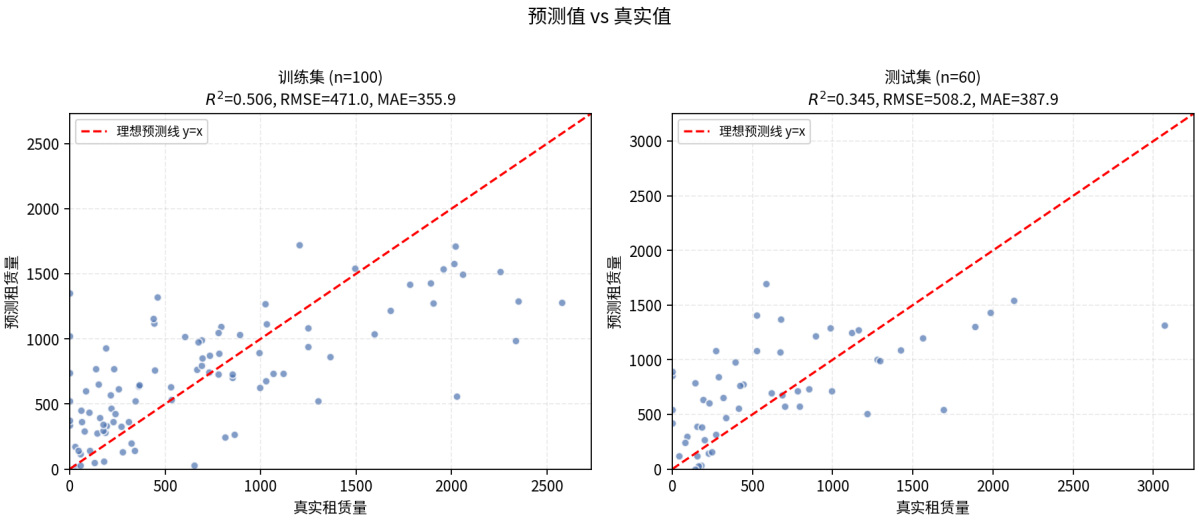


图 2 预测值 vs 真实值（左：训练集；右：测试集）

3.3 结果分析

- 拟合能力：训练集 R^2 = 0.506，模型解释了约一半的需求波动；温度、湿度等特征携带了真实的预测信息；
- 泛化能力：测试集 R^2 = 0.345、RMSE = 508.2 辆，较均值基线（RMSE = 631.2 辆）降低误差 19.5%，

MAE 由 500.6 降至 387.9 辆，模型具备实际参考价值；

- 过拟合判断：训练与测试 R^2 相差 0.161，而 5 折交叉验证 $R^2 = 0.389 \pm 0.120$（各折 0.206~0.547），测试成绩落在交叉验证的正常波动范围内；训练样本仅 100 条、参数 10 个，泛化间隙主要来自小样本下的参数估计不确定性，没有证据表明存在严重过拟合；
- 误差的主要来源：数据只含气象与日历特征，缺少“小时、星期几”等时间信息；且线性假设会把预测拉向均值，高需求时段被系统性低估。MAPE 高达 122%~130% 属“假性偏大”——约 40% 样本租赁量低于 300 辆（含 0 值），小数值样本使相对误差失去意义，本任务应主要参考 RMSE/MAE。

进一步的诊断（残差结构、分段误差、非线性证据）与改进实验见第二部分 §7、§8。

第二部分 · 超额研究

作业要求之外的延伸研究：探索性数据分析、多方法交叉验证、参数统计检验、残差诊断、改进实验与结论。

§4 探索性数据分析（补充）

建模之前我先把数据看了一遍：训练集描述性统计见表 6，变量分布见图 3，数值变量相关系数矩阵见图 4，分组统计见图 5，各特征与目标的相关系数见表 7。

表 6 训练集（n=100）数值变量描述性统计

变量	均值	标准差	最小值	25% 分位	中位数	75% 分位	最大值
温度(°C)	11.08	12.43	-15.30	0.75	12.50	21.25	32.40
湿度(%)	56.89	20.65	15.00	41.75	54.50	71.25	97.00
风速(m/s)	1.85	1.20	0.00	1.00	1.50	2.40	5.60
降雨量(mm)	0.11	0.45	0.00	0.00	0.00	0.00	2.50
降雪量(cm)	0.13	0.47	0.00	0.00	0.00	0.00	2.60
租赁自行车数量	670.93	673.57	0.00	148.75	439.50	1002.00	2577.00

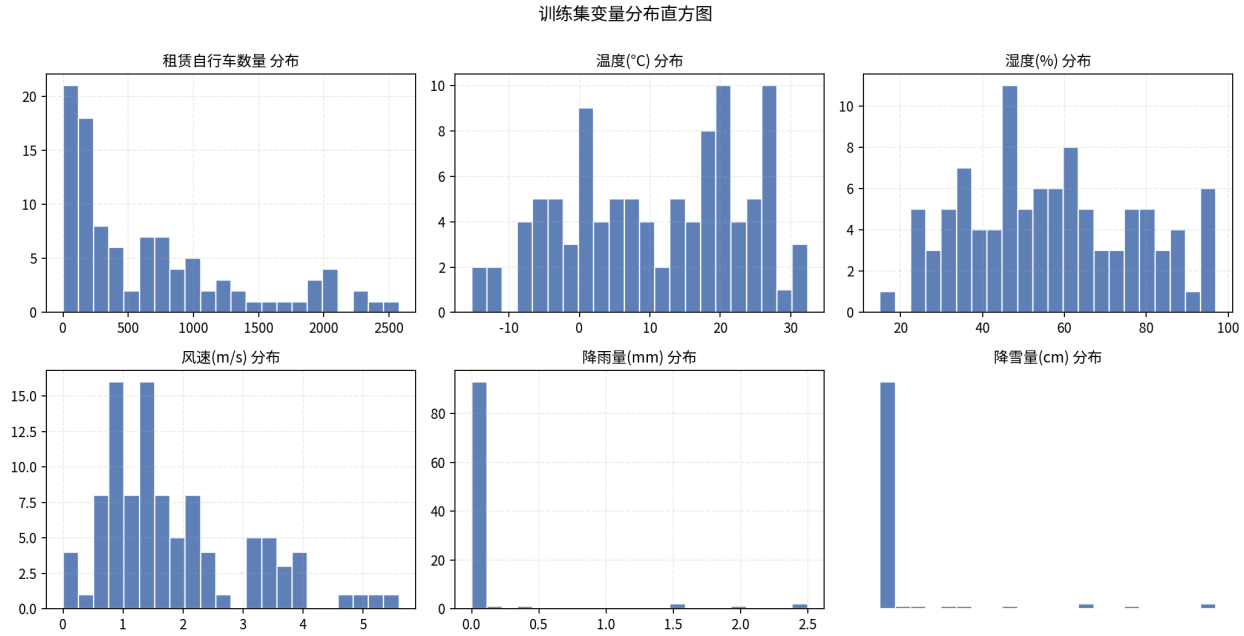


图 3 训练集变量分布直方图：目标变量明显右偏、大量低需求样本；降雨降雪高度稀疏

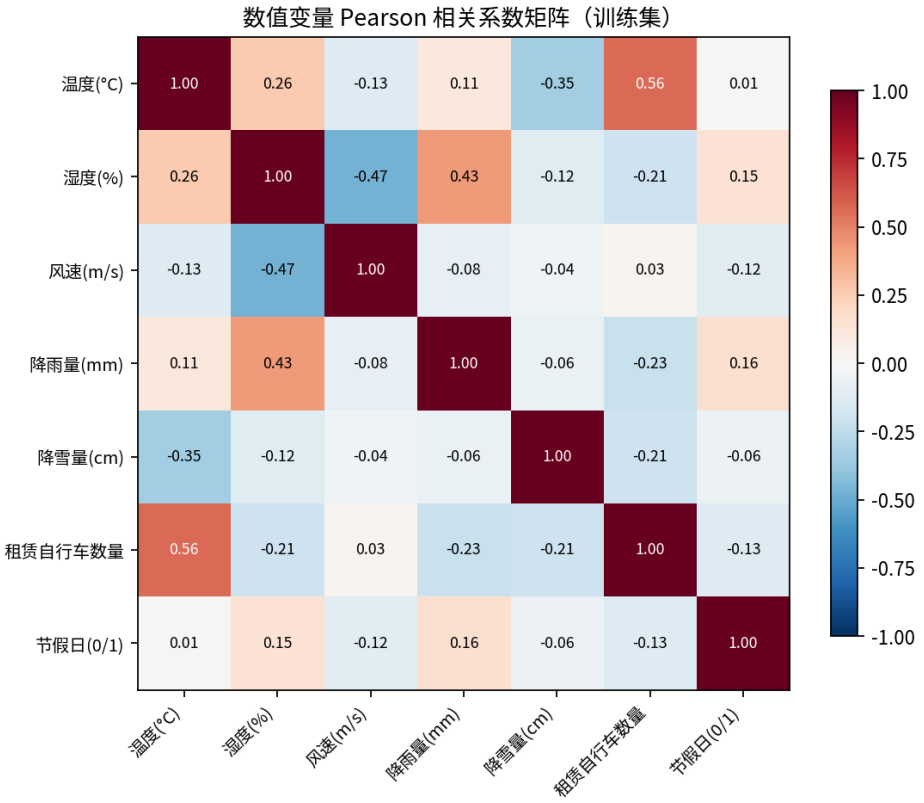


图 4 数值变量 Pearson 相关系数矩阵（训练集）：温度与需求相关性最强，降雨/湿度/降雪呈负相关

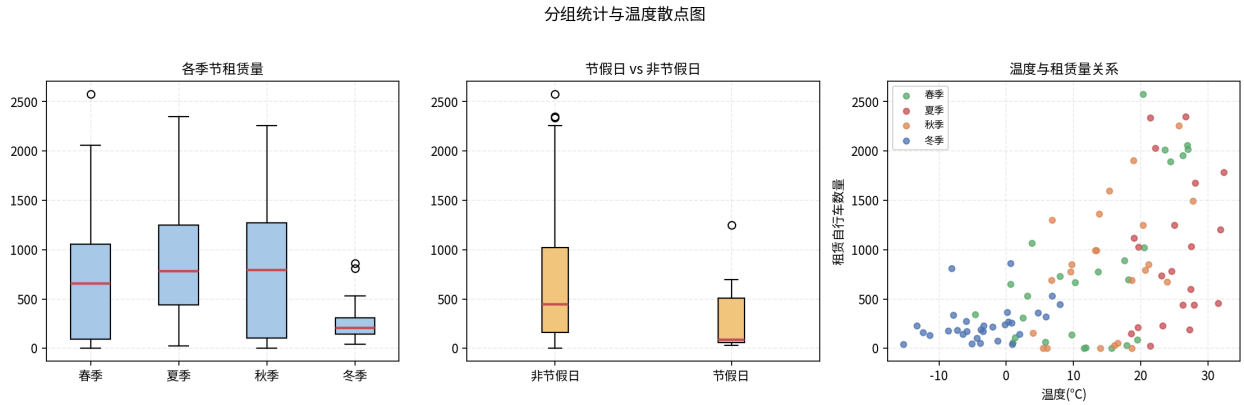


图 5 分组统计：各季节需求差异（冬季最低）、节假日需求偏低，温度与需求整体上升但非严格线性

表 7 各特征与目标变量的 Pearson 相关系数（训练集）

特征	温度	湿度	风速	降雨量	降雪量	节假日(0/1)
与租赁量相关系数 r	+0.565	-0.210	+0.027	-0.227	-0.209	-0.128

四点观察直接影响了后面的建模：

- 温度是最强单变量信号（r = 0.565）：需求随温度整体上升，但图 5 右图显示高温段（>25 °C）增速放缓甚至回落，相同温度下春季需求系统性高于夏季——提示纯线性假设可能不足，是 §8 交互项实验的直接动因；
- 降雨（r = −0.227）、湿度（r = −0.210）、降雪（r = −0.209）与需求负相关：恶劣天气抑制骑行意愿；
- 风速与需求几乎不相关（r = 0.027）：该特征信息量很低，其系数不显著（§6 将证实）；
- 低需求样本（y < 300）占比约 40%，且方差随需求增大而增大（图 5 左图夏季/秋季箱体远高于冬

季），预示残差存在异方差，评估时不应只看 R^2。

§5 三种求解方法交叉验证与数值校验

为确保最小二乘解无误，我用三种相互独立的途径求解同一问题并逐参数比较：（1）正规方程闭式解（np.linalg.solve）；（2）scikit-learn LinearRegression（底层 SVD/最小二乘分解）；（3）手写批量梯度下降（标准化特征上迭代 3 万次，目标同步中心化-缩放以加速收敛）。结果见表 8。

表 8 三种求解方法的关键参数对比（标准化尺度下；右列前者为正规方程 vs scikit-learn，后者为 vs 梯度下降）

参数	正规方程	scikit-learn	梯度下降 (3 万次)	最大绝对偏差
b*	718.527914	718.527914	718.527914	< 1e-11
w*(温度)	484.825284	484.825284	484.825284	< 1e-11
w*(湿度)	-226.817132	-226.817132	-226.817132	< 1e-11
w*(降雨)	-104.753443	-104.753443	-104.753443	< 1e-11
全部 10 个参数	—	—	—	9.3e-13 / 8.4e-12

- 三种方法得到的参数逐位一致，最大偏差处于双精度浮点舍入误差量级（9.3e-13 / 8.4e-12）；
- 参数反变换校验：用原始尺度参数直接预测与用标准化参数预测，最大差 < 5e-13，说明式 (6) 的反变换精确无误；
- 结论：闭式解推导与实现正确，且梯度下降在标准化特征与合适学习率下能收敛到同一全局最优解（损失曲线见图 1）。

§6 参数显著性检验与共线性诊断（补充）

在 OLS 框架下，我对每个参数做了 t 检验：标准误由 σ²(X^T X)^(-1) 得到（σ² = 残差平方和 / 自由度 = 246503.5，自由度 90）。结果见表 9 与图 6。

表 9 参数、标准化系数与显著性检验（n=100，自由度 90）

变量	系数（原始尺度）	标准化系数	标准误	t 值	p 值	显著性
截距 b	+1055.483	+718.53	239.525	+4.41	0.0000	***
温度 (°C)	+39.199	+484.83	7.806	+5.02	0.0000	***
湿度 (%)	-11.039	-226.82	3.355	-3.29	0.0014	**
风速 (m/s)	-61.311	-73.50	49.276	-1.24	0.2166	n.s.
降雨量 (mm)	-233.790	-104.75	125.678	-1.86	0.0661	†
降雪量 (cm)	-39.490	-18.61	117.603	-0.34	0.7378	n.s.
季节=春季	+97.676	+97.68	200.130	+0.49	0.6267	n.s.
季节=夏季	-220.914	-220.91	296.813	-0.74	0.4586	n.s.
季节=秋季	-33.487	-33.49	215.800	-0.16	0.8770	n.s.
节假日	-269.995	-270.00	213.381	-1.27	0.2090	n.s.

显著性标记：*** 表示 p<0.001，** 表示 p<0.01，* 表示 p<0.05，† 表示 p<0.1，n.s. 不显著。标准化系数 = 原始系数 × 该特征标准差，衡量“每 1 个标准差变化”的影响量级。

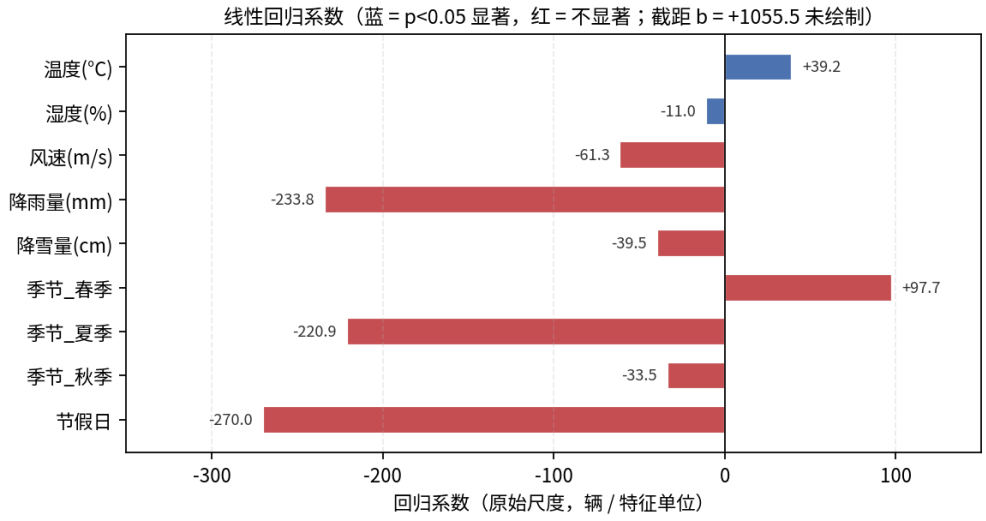


图 6 回归系数条形图（蓝 = p<0.05 显著，红 = 不显著）

参数解读：

- 截距 b = 1055.5（p<0.001）：基准场景（冬季、非节假日、0 °C、无风无雨雪）下的期望租赁量，是线性外推的数学截点，并非需求下界（见 §7 的预测压缩分析）；
- 温度 w1 = +39.199 辆/°C（p = 2.6e-6）：最强正向影响；标准化系数 +484.8 亦为最大，是唯一在两个尺度下都高度显著的连续特征；
- 湿度 w2 = −11.039 辆/%（p = 0.0014）：显著负向影响；降雨量 w4 = −233.790 辆/mm（p = 0.066，边际显著），训练集中 85% 的样本降雨为 0，其系数不确定性较大；
- 风速（p = 0.217）与降雪（p = 0.738）不显著；季节哑变量 p 值 0.46 至 0.88 不显著——在温度已进

入模型的前提下，季节的增量信息有限，“夏季”的负系数应理解为“扣除温度影响后的残余季节性”；

- 节假日 −269.995 辆（p = 0.209）：符号为负与直觉一致，但仅 7 条样本，置信区间宽（±约 424 辆），证据强度不足。

共线性诊断：用方差膨胀因子 VIF = 1/(1−R_j^2) 衡量（表 10）。全部 VIF 小于 10 的常用阈值，说明无严重多重共线性、最小二乘解稳定；“夏季”VIF = 5.93 偏高（与温度相关），是其系数不显著且需谨慎解释的直接原因。

表 10 方差膨胀因子（VIF）诊断（阈值：VIF > 10 视为严重共线性）

特征	VIF	是否共线
温度(°C)	3.78	否
湿度(%)	1.93	否
风速(m/s)	1.42	否
降雨量(mm)	1.29	否
降雪量(cm)	1.25	否
季节_春季	3.13	否
季节_夏季	5.93	轻度相关（与温度）
季节_秋季	3.35	否
节假日	1.20	否

§7 残差诊断与预测结构分析（补充）

R^2 只描述总体拟合，残差结构才能揭示系统性缺陷。图 7 为残差诊断四联图。

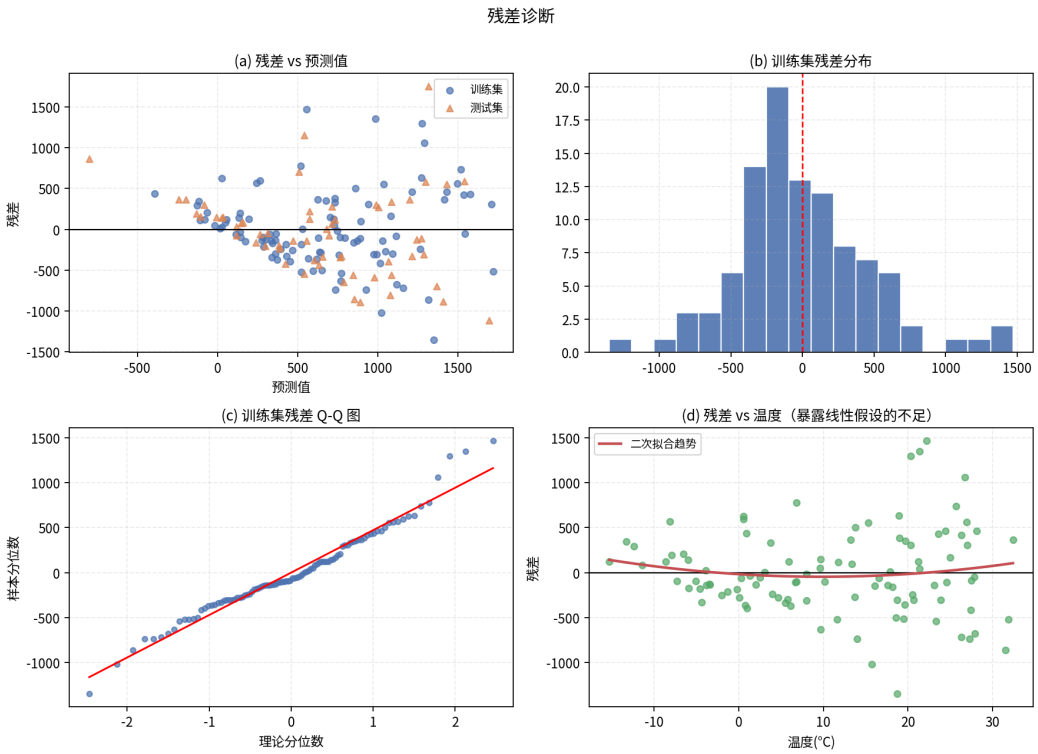


图 7 残差诊断四联图：(a) 残差 vs 预测值显示系统性正斜率；(b) 残差近似对称但尾部略厚；(c) Q-Q 图两端偏离；(d) 残差 vs 温度呈二次形态，揭示温度非线性效应

- 无全局偏置：训练残差均值 ≈ 0（1.1e-13），最小二乘解确实最小化了平方误差；
- 残差结构与异方差：图 7(a) 残差随预测值呈明显正斜率（与真实值相关 0.70），低预测段偏高估、高预测段偏低估，且误差方差随需求增大，违反同方差假设；
- 正态性轻度偏离：Shapiro-Wilk 检验 W = 0.971、p = 0.024；Jarque-Bera 检验 p = 0.008，均拒绝严格正态；偏离集中在两端，对点估计影响小，但使 t 检验与置信区间只能视为近似；
- 温度的非线性证据：图 7(d) 残差随温度呈“两端上翘、中段下凹”的二次形态——低温段高估、中温段低估、高温段再高估，与 §4 的观察一致，是 §8 引入温度二次项的直接依据。

预测结构上存在“预测压缩”：训练集预测值标准差 479.2，仅为真实值标准差 673.6 的 71.1%；预测区间 [−393.0, 1720.8] 明显窄于真实区间 [0, 2577.0]。模型倾向把预测拉向均值，对极端需求刻画不足；训练集与测试集各有 7 条样本被预测为负需求，工程落地时应做 0 截断（max(预测值, 0)）或改用对数/泊松类模型。

表 11 测试集分段误差（按真实需求量分组；“偏置” = 真实值 − 预测值，正数表示低估）

真实需求区间 (辆)	样本数	MAE (辆)	偏置 (辆)	RMSE (辆)
[0, 300)	28	335.5	-129.2	427.7
[300, 800)	16	375.2	-323.1	484.3
[800, 1500)	10	288.4	+115.8	331.2
[1500, +)	6	832.0	+832.0	959.2

分段结果清晰：需求量 ⩾ 1500 辆的 6 个样本全部被低估（平均低估 832.0 辆），中低需求段总体略被高估。高需求时段低估意味着调度不足、用户体验受损——说明单纯最小化全局 RMSE 并不等于业务最优，重要时段应加权。

§8 改进探索实验（补充）

为验证上面的诊断并寻找改进方向，我在完全相同的训练/测试划分上做了四组对照实验，结果见表 12。

表 12 改进实验结果对比（训练/测试划分与主模型完全一致）

模型	训练集 R^2	测试集 R^2	测试集 RMSE (辆)
线性回归（主模型）	0.5061	0.3450	508.2
岭回归 α = 0.1（L2 正则）	0.5061	0.3449	508.2
岭回归 α = 1.0	0.5054	0.3446	508.3
岭回归 α = 10	0.4926	0.3509	505.9
对数目标 log(1+y)	—	0.0449	613.7
对数目标 log(1+y) + Duan 校正	—	−0.8304	849.5
线性回归 + 温度² + 温度×季节交互	0.5540	0.4614	460.8

- 岭回归：α 从 0.1 到 10 均无改善（测试 R^2 变化 < 0.006）——与 §6 的无共线性结论相符：缺少共线性时，正则化只引入偏差而无方差收益；
- 对数目标：log(1+y) 训练后测试 R^2 仅 0.045。约 40% 的近零需求样本在对数尺度上权重过大，模型被迫优先拟合它们；且对数空间的无偏预测经 exp 反变换后在原始尺度上存在系统偏差，Duan 平滑校正（平滑因子 1.864）也无法挽救。说明目标变换必须与误差度量匹配，不能迷信“取对数更稳”；
- 交互与二次项：加入“温度² + 温度×季节”后，训练 R^2 由 0.506 升至 0.554、测试 R^2 由 0.345 升至 0.461（相对提升 33.7%），RMSE 由 508.2 降至 460.8 辆（降低 9.3%），训练/测试同步改善而非过拟合——验证了 §7 的诊断：需求对温度的响应是非线性的，且随季节不同；
- 方法上需要留意：所有对照实验共用同一次划分；若在测试集上反复试错并据此选模型，测试集将退化为验证集并使评估乐观化，更严谨的做法是在训练集内交叉验证选型、测试集只用于最终报告一次。

进一步改进建议（按性价比排序）：(i) 补充时间特征（小时、星期几、是否工作日、月份），共享单车需求的日内双峰节律是最强的信号源；(ii) 采用温度分段（样条/分段线性）或广义可加模型；(iii) 使用泊松/负二项回归或梯度提升树（UCI 完整数据集 8760 条上树模型 R^2 通常可达 0.85 以上）；(iv) 对高需求时段加权或使用分位数回归关注 P90 需求，直接服务调度决策；(v) 落地时对预测做 0 截断与整数化。另外说明一点：UCI 完整数据集共 8760 条，本次作业的 160 条是其子集，样本规模决定了性能上限。

§9 结论与展望

我以 Seoul Bike Sharing Demand 数据（训练 100 条 / 测试 60 条）完整走通了 “数据检查 → 特征工程 → 线性回归建模 → 多方法求解 → 训练/测试评估 → 残差诊断 → 改进探索” 的全流程，主要结论：

- 模型与求解：9 特征线性回归 $y = b + \Sigma\, w_i x_i$，参数 $b = 1055.483$ 与全部 w_i 见表 4；正规方程、scikit-learn 与手写梯度下降三种解法参数最大偏差 9.3e-13，交叉验证了求解的正确性；
- 显著性：温度（$p = 2.6e\text{-}6$）与湿度（$p = 0.0014$）显著，降雨边际显著（$p = 0.066$）；风速、降雪、季节与节假日不显著；全部 VIF < 10，无严重共线性；
- 表现：训练集 $R^2 = 0.506$ / RMSE = 471.0；测试集 $R^2 = 0.345$ / RMSE = 508.2，较均值基线降低误差 19.5%；5 折交叉验证 $R^2 = 0.389 \pm 0.120$，无严重过拟合；
- 诊断与改进：残差无异方差同分布假设下的理想形态（存在异方差与预测压缩，高需求样本平均低估 832 辆）；引入温度二次项与 “温度×季节” 交互后测试 R^2 提升至 0.461，证明非线性特征工程是行之有效的改进方向；岭回归无增益、对数目标失效，分别印证 “无严重共线性” 与 “目标变换需与误差结构匹配” 两个判断。

展望：该模型可解释、可复现、显著优于基线，适合作业务初期的参考模型与后续复杂模型的对标基线；若补充小时/星期等时间特征并采用非线性模型，预测精度仍有较大提升空间。

附录 A 完整代码（可复现）

以下是我使用的完整 Python 脚本，数据处理、训练、评估、绘图与正文全部数值都由它生成（单文件即可复现）。运行环境：Python 3.13 + numpy / pandas / scikit-learn / matplotlib / scipy。

```
# -*- coding: utf-8 -*-
"""
Seoul Bike Sharing Demand —— 线性回归课堂作业 完整可复现脚本
============================================================
流程: 读取数据 -> EDA -> 数据处理/特征工程 -> 三种方法求解线性回归
    -> 训练/测试集评估 -> 残差诊断 -> 进阶探索(岭回归/对数目标/交互项)
产出: figures/*.png  +  results/results.json (供生成PDF报告使用)

运行: python3 共享单车_线性回归_分析.py
"""
import json, os, warnings
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib import font_manager
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.model_selection import KFold, cross_val_score
from scipy import stats

warnings.filterwarnings("ignore")
np.set_printoptions(precision=4, suppress=True)
SEED = 42
rng = np.random.default_rng(SEED)

# ---------- 全局: 中文字体 ----------
for f in ["/usr/share/fonts/opentype/noto/NotoSansCJK-Regular.ttc",
          "/usr/share/fonts/opentype/noto/NotoSansCJK-Bold.ttc"]:
    font_manager.fontManager.addfont(f)
plt.rcParams["font.sans-serif"] = ["Noto Sans CJK JP", "Noto Sans CJK SC", "DejaVu Sans"]
plt.rcParams["axes.unicode_minus"] = False
plt.rcParams["figure.dpi"] = 160
plt.rcParams["savefig.bbox"] = "tight"

os.makedirs("figures", exist_ok=True)
os.makedirs("results", exist_ok=True)
R = {}   # 结果字典, 最后写入 json

# ============================================================
# 0. 读取数据
# ============================================================
tr = pd.read_csv("uploads/共享单车_训练集_100条.csv", encoding="utf-8-sig")
te = pd.read_csv("uploads/共享单车_测试集_60条.csv", encoding="utf-8-sig")
Y_COL = "租赁自行车数量"
CONT  = ["温度(°C)", "湿度(%)", "风速(m/s)", "降雨量(mm)", "降雪量(cm)"]

R["n_train"], R["n_test"], R["n_feat_raw"] = len(tr), len(te), tr.shape[1] - 1
R["missing"] = int(tr.isna().sum().sum() + te.isna().sum().sum())
R["dup_train"] = int(tr.duplicated().sum())

# 描述性统计
desc = tr[CONT + [Y_COL]].describe().T[["mean", "std", "min", "25%", "50%", "75%", "max"]]
R["desc_table"] = desc.round(2).reset_index().values.tolist()
R["desc_cols"] = ["特征"] + list(desc.columns)
R["season_counts_train"] = tr["季节"].value_counts().to_dict()
R["holiday_counts_train"] = tr["节假日"].value_counts().to_dict()

# ============================================================
# 1. 探索性数据分析
# ============================================================
# --- 图1: 目标变量与连续特征分布 ---
fig, axes = plt.subplots(2, 3, figsize=(13, 6.6))
cols_plot = [Y_COL] + CONT
for ax, c in zip(axes.ravel(), cols_plot):
    ax.hist(tr[c], bins=22, color="#4C72B0", edgecolor="white", alpha=.9)
    ax.set_title(f"{c} 分布", fontsize=11)
    ax.grid(alpha=.25, ls="--")
axes.ravel()[-1].axis("off")
fig.suptitle("训练集变量分布直方图", fontsize=13, y=1.0)
fig.tight_layout(); fig.savefig("figures/fig1_distributions.png"); plt.close(fig)

# --- 图2: 相关系数热力图 ---
num_df = tr[CONT + [Y_COL]].copy()
num_df["节假日(0/1)"] = (tr["节假日"] == "节假日").astype(int)
corr = num_df.corr()
fig, ax = plt.subplots(figsize=(7.2, 6))
```

```
im = ax.imshow(corr, cmap="RdBu_r", vmin=-1, vmax=1)
ax.set_xticks(range(len(corr))); ax.set_xticklabels(corr.columns, rotation=45, ha="right", fontsize=9)
ax.set_yticks(range(len(corr))); ax.set_yticklabels(corr.columns, fontsize=9)
for i in range(len(corr)):
    for j in range(len(corr)):
        ax.text(j, i, f"{corr.iloc[i, j]:.2f}", ha="center", va="center",
                fontsize=8, color="white" if abs(corr.iloc[i, j]) > .55 else "black")
fig.colorbar(im, shrink=.82)
ax.set_title("数值变量 Pearson 相关系数矩阵（训练集）", fontsize=12)
fig.tight_layout(); fig.savefig("figures/fig2_corr.png"); plt.close(fig)
R["corr_with_y"] = corr[Y_COL].drop(Y_COL).round(3).to_dict()

# --- 图3: 季节/节假日分组箱线图 + 温度散点 ---
fig, axes = plt.subplots(1, 3, figsize=(14, 4.4))
order = ["春季", "夏季", "秋季", "冬季"]
bp = [tr.loc[tr["季节"] == s, Y_COL].values for s in order]
axes[0].boxplot(bp, labels=order, patch_artist=True,
                boxprops=dict(facecolor="#A8C8E8"), medianprops=dict(color="#C44E52", lw=2))
axes[0].set_title("各季节租赁量", fontsize=11); axes[0].grid(alpha=.25, ls="--")
bp2 = [tr.loc[tr["节假日"] == h, Y_COL].values for h in ["非节假日", "节假日"]]
axes[1].boxplot(bp2, labels=["非节假日", "节假日"], patch_artist=True,
                boxprops=dict(facecolor="#F2C57C"), medianprops=dict(color="#C44E52", lw=2))
axes[1].set_title("节假日 vs 非节假日", fontsize=11); axes[1].grid(alpha=.25, ls="--")
cmap = {"春季": "#55A868", "夏季": "#C44E52", "秋季": "#DD8452", "冬季": "#4C72B0"}
for s in order:
    m = tr["季节"] == s
    axes[2].scatter(tr.loc[m, "温度(°C)"], tr.loc[m, Y_COL], s=22, alpha=.75, c=cmap[s], label=s)
axes[2].set_xlabel("温度(°C)"); axes[2].set_ylabel(Y_COL)
axes[2].set_title("温度与租赁量关系", fontsize=11); axes[2].legend(fontsize=8); axes[2].grid(alpha=.25, ls="--")
fig.suptitle("分组统计与温度散点图", fontsize=13, y=1.03)
fig.tight_layout(); fig.savefig("figures/fig3_groups.png"); plt.close(fig)

# ============================================================
# 2. 数据处理与特征工程
# ============================================================
def build_design(df):
    """构造设计矩阵: 连续特征 + 季节哑变量(以冬季为基准) + 节假日哑变量"""
    X = pd.DataFrame(index=df.index)
    for c in CONT:
        X[c] = df[c].astype(float)
    for s in ["春季", "夏季", "秋季"]:          # 冬季作为基准类别
        X[f"季节_{s}"] = (df["季节"] == s).astype(float)
    X["节假日"] = (df["节假日"] == "节假日").astype(float)
    return X

Xtr_raw, Xte_raw = build_design(tr), build_design(te)
ytr, yte = tr[Y_COL].astype(float).values, te[Y_COL].astype(float).values
feat_names = list(Xtr_raw.columns)
R["feature_names"] = feat_names

# 标准化(仅连续特征, 哑变量保持0/1) —— 用训练集统计量, 防止信息泄漏
mu, sd = Xtr_raw[CONT].mean().values, Xtr_raw[CONT].std(ddof=0).values
def standardize(X):
    Z = X.copy()
    Z[CONT] = (X[CONT] - mu) / sd
    return Z
Ztr, Zte = standardize(Xtr_raw), standardize(Xte_raw)
R["train_mu"], R["train_sd"] = mu.tolist(), sd.tolist()

# 条件数对比: 说明为什么要标准化
def design_matrix(X, add_one=True):
    A = X.values.astype(float)
    return np.hstack([np.ones((len(A), 1)), A]) if add_one else A

R["cond_raw"]  = float(np.linalg.cond(design_matrix(Xtr_raw)))
R["cond_std"]  = float(np.linalg.cond(design_matrix(Ztr)))
R["feature_engineering_demo"] = {
    "cols": ["原始行(节选)"] + feat_names,
    "rows": [
        list(tr.iloc[i][CONT + ["季节", "节假日"]].astype(str).values) +
        list(np.round(Xtr_raw.iloc[i].values, 2)) for i in range(3)
    ],
    "standardized_rows": [list(np.round(Ztr.iloc[i].values, 3)) for i in range(3)],
}

# ============================================================
# 3. 线性回归模型 —— 三种求解方式交叉验证
#    模型: y = b + Σ w_i x_i  (含截距)
# ============================================================
def fit_ols_normal_equation(A, y):
    """正规方程闭式解: (X^T X) w = X^T y  ->  w = (X^T X)^{-1} X^T y"""
    XtX = A.T @ A
    Xty = A.T @ y
    w = np.linalg.solve(XtX, Xty)      # 比直接求逆更稳定
    return w
```

```
Atr_std, Ate_std = design_matrix(Ztr), design_matrix(Zte)
Atr_raw, Ate_raw = design_matrix(Xtr_raw), design_matrix(Xte_raw)

w_std_ne = fit_ols_normal_equation(Atr_std, ytr)          # (1) 正规方程
lr = LinearRegression().fit(Ztr.values, ytr)              # (2) sklearn
w_std_sk = np.r_[lr.intercept_, lr.coef_]

# (3) 批量梯度下降(标准化特征上, 为加速收敛对y也做了中心化处理)
A, y = Atr_std.copy(), ytr.copy()
yb = y.mean(); ys = y.std(ddof=0)
y_scaled = (y - yb) / ys
w_gd = np.zeros(A.shape[1]); lr_gd = 0.10; iters = 30000
loss_hist = []
for t in range(iters):
    pred = A @ w_gd
    grad = 2.0 / len(A) * A.T @ (pred - y_scaled)
    w_gd -= lr_gd * grad
    if t % 50 == 0:
        loss_hist.append(float(np.mean((A @ w_gd - y_scaled) ** 2)))
# 反变换回原尺度
w_gd_orig = w_gd * ys
w_gd_orig[0] += yb

R["solver_agreement"] = {
    "normal_eq": {n: float(v) for n, v in zip(["b"] + feat_names, w_std_ne)},
    "sklearn":   {n: float(v) for n, v in zip(["b"] + feat_names, w_std_sk)},
    "gd":        {n: float(v) for n, v in zip(["b"] + feat_names, w_gd_orig)},
    "max_diff_ne_sk": float(np.max(np.abs(w_std_ne - w_std_sk))),
    "max_diff_ne_gd": float(np.max(np.abs(w_std_ne - w_gd_orig))),
    "final_loss_gd": loss_hist[-1],
    "iters": iters, "lr": lr_gd,
}
# 梯度下降收敛图
fig, ax = plt.subplots(figsize=(6.6, 4))
ax.semilogy(np.arange(len(loss_hist)) * 50, loss_hist, color="#4C72B0", lw=1.8)
ax.set_xlabel("迭代次数"); ax.set_ylabel("MSE (对数刻度)")
ax.set_title("批量梯度下降损失收敛曲线（标准化特征）", fontsize=11)
ax.grid(alpha=.3, ls="--"); fig.tight_layout()
fig.savefig("figures/fig7_gd.png"); plt.close(fig)

# ---- 最终模型: 原始尺度参数 (由标准化参数精确反变换) ----
# Z 连续列标准化: z_j = (x_j - mu_j)/sd_j ; 哑变量 z_j = x_j
# y ≈ b* + Σ_cont w*_j (x_j-mu_j)/sd_j + Σ_dummy w*_j x_j
#   = [b* - Σ_cont w*_j mu_j/sd_j] + Σ_cont (w*_j/sd_j) x_j + Σ_dummy w*_j x_j
b_std, w_std = w_std_ne[0], w_std_ne[1:]
w_raw = w_std.copy()
for k, c in enumerate(CONT):
    w_raw[k] = w_std[k] / sd[k]
b_raw = b_std - np.sum(w_std[:len(CONT)] * mu / sd)

# 数值校验: 原始尺度参数直接预测 == 标准化参数预测
pred_raw_direct = Atr_raw @ np.r_[b_raw, w_raw]
pred_std        = Atr_std @ w_std_ne
R["param_invariance_maxdiff"] = float(np.max(np.abs(pred_raw_direct - pred_std)))

R["params_raw"] = {"b": float(b_raw)}
R["params_raw"].update({n: float(v) for n, v in zip(feat_names, w_raw)})
R["params_std"] = {"b": float(b_std)}
R["params_std"].update({n: float(v) for n, v in zip(feat_names, w_std)})

# 参数标准误 / t 检验 / p 值 (OLS 推断)
n, p = Atr_raw.shape
resid = ytr - pred_raw_direct
dof = n - p
sigma2 = resid @ resid / dof
cov_w = sigma2 * np.linalg.inv(Atr_raw.T @ Atr_raw)
se = np.sqrt(np.diag(cov_w))
tvals = np.r_[b_raw, w_raw] / se
pvals = 2 * (1 - stats.t.cdf(np.abs(tvals), dof))
R["inference"] = {
    "names": ["b(截距)"] + feat_names,
    "coef": [float(b_raw)] + [float(v) for v in w_raw],
    "se":   [float(v) for v in se],
    "t":    [float(v) for v in tvals],
    "p":    [float(v) for v in pvals],
    "dof": int(dof), "sigma2": float(sigma2),
}
# 系数图
fig, ax = plt.subplots(figsize=(7.4, 4.0))
colors = ["#4C72B0" if pv < 0.05 else "#C44E52" for pv in pvals[1:]]
bars = ax.barh(feat_names[::-1], w_raw[::-1], color=colors[::-1], edgecolor="white", height=.62)
for b, v in zip(bars, w_raw[::-1]):
    ax.text(v + (6 if v >= 0 else -6), b.get_y() + b.get_height() / 2, f"{v:+.1f}",
            va="center", ha="left" if v >= 0 else "right", fontsize=8.2, color="#333333")
```

```
ax.axvline(0, color="k", lw=.8)
ax.set_xlim(-350, 150)
ax.set_xlabel("回归系数（原始尺度，辆 / 特征单位）")
ax.set_title("线性回归系数（蓝 = p<0.05 显著，红 = 不显著；截距 b = +1055.5 未绘制）", fontsize=10.5)
ax.grid(alpha=.25, ls="--", axis="x"); fig.tight_layout()
fig.savefig("figures/fig8_coef.png"); plt.close(fig)

# ============================================================
# 4. 模型评估
# ============================================================
def metrics(y_true, y_pred):
    n = len(y_true)
    ss_res = float(np.sum((y_true - y_pred) ** 2))
    ss_tot = float(np.sum((y_true - y_true.mean()) ** 2))
    r2   = 1 - ss_res / ss_tot
    rmse = float(np.sqrt(ss_res / n))
    mae  = float(np.mean(np.abs(y_true - y_pred)))
    mask = y_true > 0
    mape = float(np.mean(np.abs((y_true[mask] - y_pred[mask]) / y_true[mask])) * 100)
    return {"R2": r2, "RMSE": rmse, "MAE": mae, "MAPE": mape}

pred_tr = Atr_raw @ np.r_[b_raw, w_raw]
pred_te = Ate_raw @ np.r_[b_raw, w_raw]
m_tr, m_te = metrics(ytr, pred_tr), metrics(yte, pred_te)

# 调整 R²
def adj_r2(r2, n, p): return 1 - (1 - r2) * (n - 1) / (n - p - 1)
m_tr["R2_adj"] = adj_r2(m_tr["R2"], len(ytr), p - 1)
m_te["R2_adj"] = adj_r2(m_te["R2"], len(yte), p - 1)
R["metrics_train"], R["metrics_test"] = m_tr, m_te

# 基线: 用训练集均值预测
R["baseline_test"] = metrics(yte, np.full_like(yte, ytr.mean()))
# 仅截距模型
R["baseline_train"] = metrics(ytr, np.full_like(ytr, ytr.mean()))

# 5折交叉验证 (训练集内部)
kf = KFold(n_splits=5, shuffle=True, random_state=SEED)
cv_scores = cross_val_score(LinearRegression(), Ztr.values, ytr, cv=kf, scoring="r2")
cv_mse = -cross_val_score(LinearRegression(), Ztr.values, ytr, cv=kf, scoring="neg_mean_squared_error")
R["cv"] = {"R2_folds": [float(v) for v in cv_scores], "R2_mean": float(cv_scores.mean()),
           "R2_std": float(cv_scores.std()), "RMSE_folds": [float(np.sqrt(v)) for v in cv_mse],
           "RMSE_mean": float(np.sqrt(cv_mse).mean())}

# 图5: 预测 vs 真实
fig, axes = plt.subplots(1, 2, figsize=(11.4, 4.8))
for ax, y_t, y_p, ttl, m in [(axes[0], ytr, pred_tr, "训练集 (n=100)", m_tr),
                             (axes[1], yte, pred_te, "测试集 (n=60)", m_te)]:
    ax.scatter(y_t, y_p, s=26, alpha=.7, color="#4C72B0", edgecolor="white")
    lim = [0, max(y_t.max(), y_p.max()) * 1.06]
    ax.plot(lim, lim, "r--", lw=1.5, label="理想预测线 y=x")
    ax.set_xlim(lim); ax.set_ylim(lim)
    ax.set_xlabel("真实租赁量"); ax.set_ylabel("预测租赁量")
    ax.set_title(f"{ttl}\n$R^2$={m['R2']:.3f}, RMSE={m['RMSE']:.1f}, MAE={m['MAE']:.1f}", fontsize=10.5)
    ax.legend(fontsize=8.5, loc="upper left"); ax.grid(alpha=.25, ls="--")
fig.suptitle("预测值 vs 真实值", fontsize=13, y=1.02)
fig.tight_layout(); fig.savefig("figures/fig5_pred_vs_true.png"); plt.close(fig)

# 图6: 残差诊断
res_tr, res_te = ytr - pred_tr, yte - pred_te
fig, axes = plt.subplots(2, 2, figsize=(11.2, 8.0))
axes[0,0].scatter(pred_tr, res_tr, s=24, alpha=.7, color="#4C72B0", label="训练集")
axes[0,0].scatter(pred_te, res_te, s=24, alpha=.7, color="#DD8452", marker="^", label="测试集")
axes[0,0].axhline(0, color="k", lw=1)
axes[0,0].set_xlabel("预测值"); axes[0,0].set_ylabel("残差")
axes[0,0].set_title("(a) 残差 vs 预测值", fontsize=11); axes[0,0].legend(fontsize=8.5); axes[0,0].grid(alpha=.25, ls="--")
axes[0,1].hist(res_tr, bins=18, color="#4C72B0", edgecolor="white", alpha=.9)
axes[0,1].axvline(0, color="r", lw=1.2, ls="--")
axes[0,1].set_title("(b) 训练集残差分布", fontsize=11); axes[0,1].grid(alpha=.25, ls="--")
stats.probplot(res_tr, dist="norm", plot=axes[1,0])
axes[1,0].get_lines()[0].set(marker="o", ms=4, alpha=.7, color="#4C72B0")
axes[1,0].get_lines()[1].set(color="r", lw=1.4)
axes[1,0].set_title("(c) 训练集残差 Q-Q 图", fontsize=11); axes[1,0].grid(alpha=.25, ls="--")
axes[1,0].set_xlabel("理论分位数"); axes[1,0].set_ylabel("样本分位数")
axes[1,1].scatter(tr["温度(°C)"], res_tr, s=24, alpha=.7, color="#55A868")
axes[1,1].axhline(0, color="k", lw=1)
xs = np.linspace(tr["温度(°C)"].min(), tr["温度(°C)"].max(), 50)
coef_t = np.polyfit(tr["温度(°C)"], res_tr, 2)
axes[1,1].plot(xs, np.polyval(coef_t, xs), color="#C44E52", lw=2, label="二次拟合趋势")
axes[1,1].set_xlabel("温度(°C)"); axes[1,1].set_ylabel("残差")
axes[1,1].set_title("(d) 残差 vs 温度（暴露线性假设的不足）", fontsize=11)
axes[1,1].legend(fontsize=8.5); axes[1,1].grid(alpha=.25, ls="--")
fig.suptitle("残差诊断", fontsize=13, y=1.0)
fig.tight_layout(); fig.savefig("figures/fig6_residuals.png"); plt.close(fig)
```

```
# 残差正态性 & 异方差检验
sh_w, sh_p = stats.shapiro(res_tr)
bp_stat, bp_p = stats.jarque_bera(res_tr)[0], stats.jarque_bera(res_tr)[1]
R["residual_tests"] = {"shapiro_W": float(sh_w), "shapiro_p": float(sh_p),
                       "jb_stat": float(bp_stat), "jb_p": float(bp_p),
                       "resid_mean": float(res_tr.mean()), "resid_std": float(res_tr.std(ddof=1))}
# 残差与真实值的相关(检测系统性偏差)
R["resid_corr_y"] = float(np.corrcoef(res_tr, ytr)[0, 1])

# ============================================================
# 5. 进阶探索
# ============================================================
adv = []
# (a) 岭回归
for a in [0.1, 1.0, 10.0]:
    rg = Ridge(alpha=a).fit(Ztr.values, ytr)
    adv.append({"name": f"岭回归 alpha={a}",
                "train_R2": metrics(ytr, rg.predict(Ztr.values))["R2"],
                "test_R2": metrics(yte, rg.predict(Zte.values))["R2"],
                "test_RMSE": metrics(yte, rg.predict(Zte.values))["RMSE"]})
# (b) 对数目标 (log(1+y)), 反变换需校正
ylog = np.log1p(ytr)
lr_log = LinearRegression().fit(Ztr.values, ylog)
p_te_log = np.expm1(lr_log.predict(Zte.values))
adv.append({"name": "对数目标 log(1+y)", "train_R2": None,
            "test_R2": metrics(yte, p_te_log)["R2"], "test_RMSE": metrics(yte, p_te_log)["RMSE"]})
# (c) 增加 温度² 与 温度×季节交互
Ztr_x = Ztr.copy(); Zte_x = Zte.copy()
for D in (Ztr_x, Zte_x):
    D["温度²"] = D["温度(°C)"] ** 2
for s in ["春季", "夏季", "秋季"]:
    Ztr_x[f"温度×{s}"] = Ztr_x["温度(°C)"] * Ztr_x[f"季节_{s}"]
    Zte_x[f"温度×{s}"] = Zte_x["温度(°C)"] * Zte_x[f"季节_{s}"]
lr_x = LinearRegression().fit(Ztr_x.values, ytr)
adv.append({"name": "线性回归 + 温度² + 温度×季节交互",
            "train_R2": metrics(ytr, lr_x.predict(Ztr_x.values))["R2"],
            "test_R2": metrics(yte, lr_x.predict(Zte_x.values))["R2"],
            "test_RMSE": metrics(yte, lr_x.predict(Zte_x.values))["RMSE"]})
# (b2) 对数目标 + Duan 平滑(涂抹)校正
res_log = ylog - lr_log.predict(Ztr.values)
smear = float(np.mean(np.exp(res_log)))
p_te_duan = np.expm1(lr_log.predict(Zte.values) + np.log(smear))
adv.append({"name": "对数目标 log(1+y) + Duan校正", "train_R2": None,
            "test_R2": metrics(yte, p_te_duan)["R2"], "test_RMSE": metrics(yte, p_te_duan)["RMSE"]})
R["duan_smear"] = smear
R["advanced"] = adv

# 方差膨胀因子 VIF（共线性诊断）
vif = {}
D = Xtr_raw.values.astype(float)
for j, c in enumerate(feat_names):
    others = np.delete(D, j, axis=1)
    r2j = LinearRegression().fit(others, D[:, j]).score(others, D[:, j])
    vif[c] = float(1.0 / max(1e-12, 1 - r2j))
R["vif"] = vif

# 预测值分布 vs 真实值分布（观察 “均值回归” 压缩效应）
R["pred_stats"] = {"true_std": float(ytr.std(ddof=1)), "pred_std_train": float(pred_tr.std(ddof=1)),
                   "true_range": [float(ytr.min()), float(ytr.max())],
                   "pred_range_train": [float(pred_tr.min()), float(pred_tr.max())],
                   "corr_true_pred_train": float(np.corrcoef(ytr, pred_tr)[0, 1]),
                   "corr_true_pred_test": float(np.corrcoef(yte, pred_te)[0, 1]),
                   "pred_negative_count_train": int((pred_tr < 0).sum()),
                   "pred_negative_count_test": int((pred_te < 0).sum())}
# 分区间误差（检查误差是否随需求水平增大）
bins_y = [0, 300, 800, 1500, np.inf]
seg = []
for lo, hi in zip(bins_y[:-1], bins_y[1:]):
    m = (yte >= lo) & (yte < hi)
    if m.sum() > 0:
        seg.append({"range": f"[{int(lo)}, {int(hi) if np.isfinite(hi) else '+'})", "n": int(m.sum()),
                    "MAE": float(np.mean(np.abs(res_te[m]))), "bias": float(np.mean(res_te[m])),
                    "RMSE": float(np.sqrt(np.mean(res_te[m] ** 2)))})
R["segment_test"] = seg

# ============================================================
# 6. 导出预测结果与结果 JSON
# ============================================================
out_tr = tr.copy(); out_tr["预测值"] = np.round(pred_tr, 1); out_tr["残差"] = np.round(res_tr, 1)
out_te = te.copy(); out_te["预测值"] = np.round(pred_te, 1); out_te["残差"] = np.round(res_te, 1)
out_tr.to_csv("results/训练集_预测与残差.csv", index=False, encoding="utf-8-sig")
out_te.to_csv("results/测试集_预测与残差.csv", index=False, encoding="utf-8-sig")

with open("results/results.json", "w", encoding="utf-8") as f:
    json.dump(R, f, ensure_ascii=False, indent=1)
```

```
print("=" * 66)
print("求解一致性: 正规方程 vs sklearn 最大差 =", f"{R['solver_agreement']['max_diff_ne_sk']:.2e}",
      "| vs 梯度下降 =", f"{R['solver_agreement']['max_diff_ne_gd']:.2e}")
print("参数反变换校验 |Δpred|max =", f"{R['param_invariance_maxdiff']:.2e}")
print(f"训练集: R2={m_tr['R2']:.4f}  RMSE={m_tr['RMSE']:.2f}  MAE={m_tr['MAE']:.2f}  MAPE={m_tr['MAPE']:.2f}%")
print(f"测试集: R2={m_te['R2']:.4f}  RMSE={m_te['RMSE']:.2f}  MAE={m_te['MAE']:.2f}  MAPE={m_te['MAPE']:.2f}%")
print("原始尺度参数:", {k: round(v, 3) for k, v in R["params_raw"].items()})
print("5折CV R2:", [round(v, 3) for v in R["cv"]["R2_folds"]], "均值", round(R["cv"]["R2_mean"], 3))
print("VIF:", {k: round(v, 2) for k, v in R["vif"].items()})
print("预测区间(训练):", R["pred_stats"]["pred_range_train"], "真实区间:", R["pred_stats"]["true_range"],
      "| pred std/truestd =", round(R["pred_stats"]["pred_std_train"] / R["pred_stats"]["true_std"], 3))
print("分段误差(测试集):", R["segment_test"])
print("=" * 66)
print("图: ", sorted(os.listdir("figures")))
```

数据来源：

UCI Machine Learning Repository, Seoul Bike Sharing Demand —— https://archive.ics.uci.edu/dataset/560/seoul+bike+sharing+demand（作业提供其子集）。